

betting-ad-blocker

betting-ad-blocker is an offline computer-vision pipeline for detecting betting-related advertising boards and broadcast overlays in sports video, then hiding them with a flat, board-coloured fill that avoids covering people. The project combines YOLO segmentation, synthetic data generation, real-annotation review, tracker-based video processing, and optional TFLite export for deployment.

What the project does

The system is designed for sports broadcasts where betting ads appear on:

- physical sideline or perimeter boards
- broadcast overlay graphics and on-screen branding

The pipeline trains a segmentation model to detect these regions, then renders a masked replacement in the final video. The implementation is intentionally conservative:

- people are detected and protected from being painted over
- the fill is a frozen flat colour rather than a learned inpainting result
- the tracker carries panels through short detection gaps to reduce flicker
- brand names are not treated as model classes; they are treated as data folders and verification signals

Project goals

- detect betting-ad regions in a general, brand-agnostic way
- keep the model robust across different sports, camera angles, and board styles
- avoid false positives on non-betting content
- produce a clean output video without black boxes or blur
- support both research/annotation workflows and an unattended production-style run

Core concepts

1. Classes

The model is trained on three semantic classes:

- ``person``
- ``betting_board``
- ``betting_overlay``

Brand names such as ``bet365``, ``betano``, ``fanduel``, ``stake``, and ``polymarket`` are not classes. They are simply folders of example logos and optional brand metadata.

2. Labeling convention

Real annotations are expected to follow a simple, important rule:

- annotate one polygon per physical ad strip, not one polygon per logo
- do not annotate people by hand; they are auto-labelled later
- keep the polygon tightly over the actual board/overlay surface without spilling into adjacent unrelated content

3. Main pipeline stages

The workflow is:

- collect input videos and logos
- extract frames from videos
- create or review real labels for boards/overlays
- generate synthetic training samples
- auto-label people
- train or fine-tune the segmentation model
- run the video processor to generate blocked output
- optionally export the model to TFLite

Repository layout

- ``scripts/`` – all training, annotation, augmentation, rendering, and evaluation logic
- ``data/`` – raw videos, extracted frames, annotations, logos, negatives, synthetic data, and dataset manifests
- ``input/`` – user-supplied videos and logos
- ``models/`` – trained checkpoints
- ``outputs/`` – rendered videos, previews, and debug outputs
- ``notebooks/`` – notebook-based workflows, especially training and experiment runs
- ``trackers/`` – ByteTrack-style tracker configuration for panel persistence
- ``config.yaml`` – main configuration file for paths, training, inference, and rendering
- ``config.local.yaml`` – optional local overrides written by notebooks or local runs

Setup

Requirements

- Python 3.10+
- ``ffmpeg`` on ``PATH``
- a CUDA-capable NVIDIA GPU for training and inference

Install dependencies

```
pip install -r requirements.txt
```

If PyTorch/CUDA is not already matching your system, install a compatible torch build first:

```
pip install torch --index-url https://download.pytorch.org/whl/cu121
pip install -r requirements.txt
```

Check your GPU visibility:

```
nvidia-smi
```

Configuration

The project uses `config.yaml` as its single source of truth. Most scripts load it through `scripts/common.py`.

Important areas in `config.yaml`:

- `paths` – directories for videos, frames, annotations, dataset, outputs, and models
- `extract` – frame extraction strategy and FPS settings
- `synthetic` – synthetic-logo generation parameters
- `autolabel` – person auto-labelling settings
- `prelabel` – zero-shot or model-assisted board draft generation
- `assemble` – dataset assembly and validation split behavior
- `train` – training settings and checkpoint naming
- `inference` – detection confidence, tracker settings, panel rendering, and occlusion handling

A notebook can also write `config.local.yaml` to override local values without changing the defaults.

Quickstart

Option A: notebook workflow

The notebook in `notebooks/pipeline.ipynb` is the main control surface for the end-to-end workflow. In practice it is used by:

- placing input videos in `input/videos/`
- placing logo images under `input/logos/`
- opening the notebook and running the workflow

The notebook automatically skips steps that are already complete and reuses existing outputs where possible.

Option B: command-line workflow

The same flow is available through scripts:

```
python scripts/extract_frames.py
python scripts/prelabel_boards.py
python scripts/generate_synthetic.py
python scripts/assemble_dataset.py
python scripts/autolabel_persons.py
python scripts/train.py
python scripts/process_video.py --input data/raw_videos/.mp4
```

Training workflow

1. Prepare input data

Put source videos in `data/raw\_videos/` (or the configured input path) and add transparent PNG logos to a brand folder such as:

- `data/logos/bet365/`
- `data/logos/fanduel/`
- `data/logos/betano/`

2. Extract frames

```
python scripts/extract_frames.py
```

This writes frames into `data/frames/`.

3. Create or review real annotations

You can either:

- manually annotate frames in CVAT/Roboflow and place them under `data/annotations\_real/`
- use `scripts/prelabel\_boards.py` to generate draft polygons for review

The project strongly prefers a small number of carefully reviewed real frames over a larger noisy set.

4. Generate synthetic data

```
python scripts/generate_synthetic.py
```

Synthetic samples are produced by pasting logos onto real-looking backgrounds. This helps the detector learn board-like geometry and branding patterns even when real labels are sparse.

5. Auto-label people

```
python scripts/assemble_dataset.py
python scripts/autolabel_persons.py
```

`assemble\_dataset.py` builds the dataset structure and the train/validation split. `autolabel\_persons.py` fills in the `person` class using a pretrained segmentation model.

6. Train the model

```
python scripts/train.py
```

By default this trains a YOLO11-seg model from the configured checkpoint. Fine-tuning from an existing checkpoint is also supported:

```
python scripts/train.py --weights models/best.pt --epochs 40 --lr0 0.001
```

The trained checkpoint is copied to `models/best.pt`.

Inference and video processing

Run the blocker on a video

```
python scripts/process_video.py --input data/raw_videos/.mp4
```

Output files are written under `outputs/` by default. A debug mode is available:

```
python scripts/process_video.py --input data/raw_videos/.mp4 --debug
```

This produces a side-by-side diagnostic view showing:

- raw detections
- person masks and occlusion handling
- the panel lifecycle and propagated geometry
- the final rendered output

Notes on inference behaviour

The inference stage is intentionally low-confidence at the detector level so the tracker can keep panels alive through short gaps. Creating a panel still requires more evidence than a single weak detection. This design reduces blinking and improves robustness, especially during motion-blurred pans.

Key scripts

- `scripts/extract\_frames.py` – sample frames from raw videos
- `scripts/prelabel\_boards.py` – draft candidate board polygons for human review
- `scripts/generate\_synthetic.py` – synthesize training images with pasted logos
- `scripts/assemble\_dataset.py` – build the final dataset structure and splits
- `scripts/autolabel\_persons.py` – add person segmentation labels

- ``scripts/train.py`` – train or fine-tune the detection model
- ``scripts/process_video.py`` – render the blocked video output
- ``scripts/run_all.py`` – run a longer unattended pipeline from label generation through rendering
- ``scripts/run_automatic.py`` – automatic end-to-end processing for a selected input video
- ``scripts/eval_holdout.py`` – evaluate on a held-out video or clip
- ``scripts/export_tflite.py`` – export the trained checkpoint to ONNX/TFLite
- ``scripts/diagnose_panel_height.py`` and ``scripts/diagnose_rendered_panel.py`` – inspect geometry and rendering problems

Data conventions and quality expectations

To get good results, keep the following rules in mind:

- prefer a smaller set of high-quality real labels over many weak ones
- annotate whole ad strips, not individual logos
- keep negatives in ``data/negatives/`` for non-betting content
- review synthetic and prelabel outputs before trusting them for training
- use fine-tune workflows after adding new brands or videos instead of always retraining from scratch

Outputs you should expect

After a successful run, the project produces:

- ``models/best.pt`` – trained or fine-tuned weights
- ``data/dataset/`` – assembled training data
- ``runs/segment/...`` – Ultralytics training output
- ``outputs/*.mp4`` – blocked videos
- ``outputs/holdout_preview/`` – evaluation previews
- ``outputs/chunks/`` – chunked render outputs used by unattended runs

Troubleshooting

Common issues:

- ``ffmpeg`` missing: install it and ensure it is on your ``PATH``
- no GPU detected: verify CUDA/PyTorch installation and ``nvidia-smi``
- empty or missing dataset folders: run the earlier assembly/prelabel steps first
- poor detection quality: review labels, add more real frames, and consider fine-tuning from ``models/best.pt``

Summary

This repository is a full offline pipeline for training a segmentation model that detects betting-ad regions in sports broadcasts and hides them in a controlled, non-destructive way. It is built around a practical annotation loop, synthetic data generation, tracker-based inference, and a flexible training workflow that can be run either interactively in notebooks or unattended from the command line.